

SiliQun: A Tensor-Network Gymnasium Environment for Deep Reinforcement Learning-Based Control of Silicon Spin Qubits

Raad Alshehri^{a,*}

^a*Faculty of Computing and Information Technology, King Abdulaziz University, Jeddah 21589, Saudi Arabia*

Abstract

Controlling silicon spin qubits with high fidelity under realistic noise conditions is one of the central challenges in scaling quantum processors. Deep reinforcement learning (DRL) offers a promising data-driven approach to discovering robust control policies, yet researchers currently lack simulation environments that are both physically grounded in silicon device physics and natively compatible with modern DRL training pipelines. We present SiliQun, an open-source Python package that fills this gap by combining a tensor-network simulation core—based on Matrix Product States (MPS) and Matrix Product Operators (MPO)—with a native OpenAI Gymnasium interface. SiliQun ships with calibrated device profiles for three major silicon spin qubit architectures (phosphorus donor, SiMOS quantum dot, and gate-all-around nanowire), implements physically motivated noise channels (T_1 relaxation, T_2^* dephasing, and $1/f$ charge noise), and supports multiple computational backends (NumPy, JAX, CUDA). Benchmarks on an Intel Xeon E5-2695 v2 processor demonstrate sustained throughput of approximately 91 000 simulation steps per second from 2 to 16 qubits, exhibiting the flat scaling characteristic of tensor-network methods. Comparative benchmarks show that SiliQun achieves 6–29 $\times$ higher throughput than Qiskit Aer’s statevector simulator across 2–14 qubit systems. Gate fidelity validation confirms machine-precision accuracy ($\leq 2.2 \times 10^{-16}$) against analytical expectations for all implemented operations. SiliQun is freely available under the MIT license.

Keywords: Quantum Computing, Silicon Spin Qubits, Tensor Networks, Deep Reinforcement Learning, Gymnasium, Quantum Control

*Corresponding author

Email address: `raalshehri0468@stu.kau.edu.sa` (Raad Alshehri)

Required Metadata

Current code version

Nr.	Code metadata description	
C1	Current code version	v1.0.0
C2	Permanent link to code/repository	https://github.com/raad-alshehri/siliqun
C3	Permanent link to Reproducible Capsule	–
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python 3.8+, NumPy, JAX, CuPy, Gymnasium
C7	Compilation requirements, operating environments & dependencies	Python ≥ 3.8 ; Linux, macOS, or Windows; NumPy ≥ 1.21
C8	Link to developer documentation	https://github.com/raad-alshehri/siliqun#readme
C9	Support email for questions	raalshehri0468@stu.kau.edu.sa

Table 1: Code metadata.

1. Motivation and significance

Silicon spin qubits are widely regarded as one of the most promising platforms for building large-scale quantum processors. Their nanometre-scale footprint, compatibility with industrial CMOS fabrication, and demonstrated coherence times exceeding 30 seconds for phosphorus donors in isotopically enriched ^{28}Si substrates make them attractive candidates for fault-tolerant quantum computing [1, 2]. Recent experimental milestones have demonstrated single-qubit gate fidelities above 99.9% in SiMOS quantum dots [4] and two-qubit gate fidelities above 99% across multiple silicon platforms [5, 6, 7, 8], bringing the platform closer to the surface code error correction threshold.

Despite this hardware progress, achieving consistently high-fidelity control remains difficult in practice. Silicon spin qubits are sensitive to charge noise arising from interface traps and defects, nuclear spin fluctuations from residual ^{29}Si , and crosstalk between neighbouring dots [4, 12, 11]. Traditional optimal control approaches—such as GRAPE and Krotov methods—require

differentiable models and struggle with the stochastic, non-stationary noise profiles encountered in real devices [13].

Deep reinforcement learning (DRL) has emerged as a compelling alternative. By interacting with a simulated environment through trial and error, DRL agents can discover control strategies that are inherently robust to the noise distributions they encounter during training [14, 15, 16, 17]. However, applying DRL to silicon spin qubits imposes a specific set of requirements on the simulation environment that existing tools do not jointly satisfy.

First, the simulator must be fast enough to support the millions of environment interactions that modern DRL algorithms demand. Device-level TCAD tools such as QTCAD [18] provide exquisite physical detail through finite-element modelling, but a single simulation step can take seconds—far too slow for DRL training loops. Second, the simulator must capture the essential physics of silicon spin qubits, including device-specific coherence times and noise channels, so that learned policies transfer meaningfully to real hardware. General-purpose circuit simulators like Qiskit Aer [19] or Cirq [20] model abstract qubits without silicon-specific device parameters. Third, the simulator must expose a standard reinforcement learning interface so that researchers can leverage the rich ecosystem of existing DRL libraries (Stable-Baselines3 [23], Ray RLlib [24]) without writing custom glue code. The recently released SpinPulse package [21] addresses the physics modelling gap with sophisticated non-Markovian noise models for spin qubits, but it does not provide a Gymnasium-compatible interface for DRL training. PennyLane’s pulse module [22] supports differentiable pulse simulation but targets gradient-based optimization rather than the sample-based exploration that DRL requires. Table 2 summarizes this landscape.

Table 2: Comparison of SiliQun with existing quantum simulation tools.

Feature	SiliQun	Qiskit Aer	QuTiP	SpinPulse	PennyLane
Silicon device profiles	✓	×	×	✓	×
Gymnasium interface	✓	×	×	×	×
Tensor network core	✓	×	×	×	×
Multi-backend (CPU/GPU)	✓	✓	×	×	✓
Non-Markovian noise	×	×	✓	✓	×
DRL-native design	✓	×	×	×	×

We developed SiliQun to fill this specific niche. SiliQun is, to our knowledge, the first quantum simulator that combines (i) a tensor-network simulation core designed for throughput rather than maximum physical fidelity, (ii) built-in device profiles calibrated to three major silicon spin qubit architectures with parameters validated against the experimental literature, and

(iii) a native Gymnasium environment wrapper that makes the simulator immediately usable with any standard DRL library.

2. Software description

SiliQun is implemented in approximately 3 400 lines of Python and is organized into four loosely coupled layers. This modular design allows researchers to swap computational backends, add new device profiles, or modify the reward structure without touching the core simulation logic.

2.1. Software architecture

Figure 1 illustrates the layered architecture. Each layer communicates with its neighbours through well-defined interfaces, and the entire stack is orchestrated by the `SiliQunEnv` class at the top.

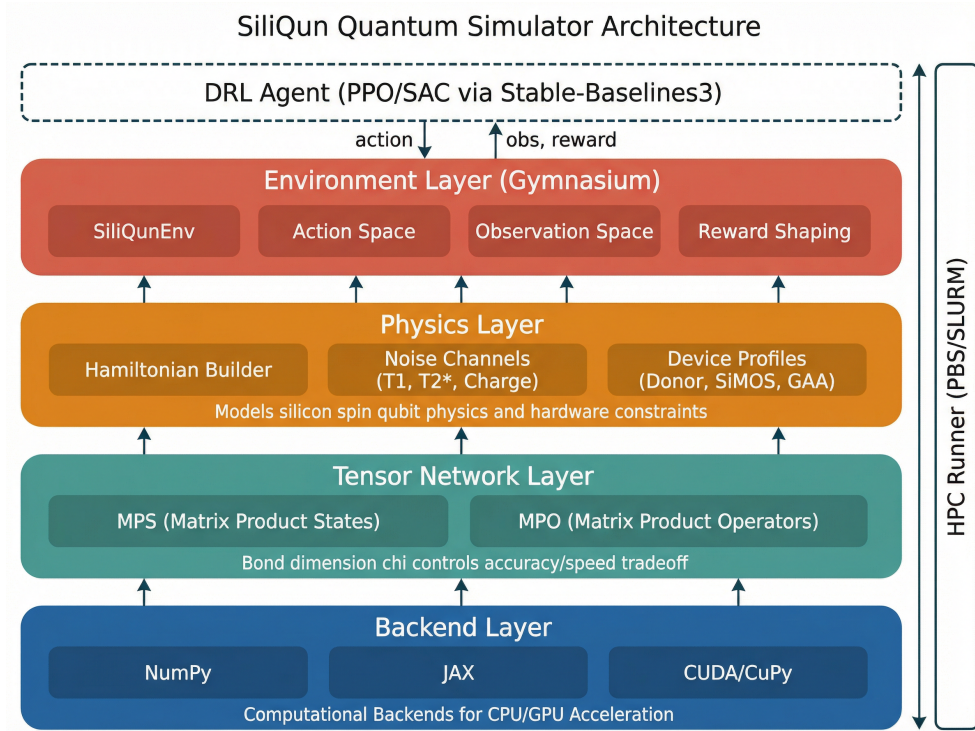


Figure 1: Layered architecture of SiliQun. The Backend Layer provides numerical primitives across CPU and GPU platforms. The Tensor Network Layer represents quantum states as MPS and applies gates as MPO contractions. The Physics Layer encodes silicon-specific device parameters and noise models. The Environment Layer exposes the full simulation as a standard Gymnasium environment for DRL training. The HPC Runner spans all layers, enabling automated job submission to PBS/SLURM clusters.

Backend Layer. The lowest layer abstracts all numerical operations behind a common interface. Three backends are currently implemented: a NumPy backend for broad compatibility, a JAX backend that enables just-in-time compilation and automatic differentiation, and a CUDA backend (via CuPy) for direct GPU acceleration. Switching backends requires changing a single configuration parameter.

Tensor Network Layer. Quantum states are represented as Matrix Product States (MPS), and quantum operations are represented as Matrix Product Operators (MPO). This representation is particularly well suited to the linear chain topology of most silicon spin qubit arrays, where nearest-neighbour entanglement dominates. The maximum bond dimension χ serves as a tunable knob that trades simulation accuracy for speed: small χ values enable extremely fast approximate simulation, while larger values recover the exact state-vector result. Gate application proceeds by contracting the MPO representation of each gate with the current MPS, followed by optional singular value decomposition (SVD) truncation to enforce the bond dimension constraint.

Physics Layer. This layer encodes the specific characteristics of silicon spin qubit hardware. It provides three predefined device profiles, summarized in Table 3, with parameters drawn from the experimental literature. The phosphorus donor profile is calibrated to the measurements of Muhonen et al. [2], who demonstrated $T_1 = 30$ s and $T_2^* = 0.5$ ms for electron spins in isotopically enriched ^{28}Si . The SiMOS quantum dot profile follows Yoneda et al. [4], who reported $T_2^* = 20$ μs and single-qubit gate fidelities exceeding 99.9%. The gate-all-around (GAA) nanowire profile is based on projections from FinFET experiments by Geyer et al. [10]. Noise is modelled through physically motivated completely positive trace-preserving (CPTP) channels applied after each ideal gate operation: amplitude damping (parameterized by T_1 and gate duration), phase damping (parameterized by T_2^*), and a stochastic $1/f$ charge noise generator that perturbs the qubit detuning with amplitudes consistent with the measurements reviewed by Connors et al. [11].

Environment Layer. The `SiliQunEnv` class implements the `gymnasium.Env` interface. At each step, the DRL agent selects a discrete action corresponding to a quantum gate (drawn from the set $\{R_x, R_y, R_z, \text{CNOT}, H, I\}$ with parameterized rotation angles). The environment applies the gate to the MPS state, applies noise, and returns an observation vector consisting of Pauli expectation values ($\langle X_i \rangle, \langle Y_i \rangle, \langle Z_i \rangle$) for each qubit. The reward is computed as the state fidelity with respect to a configurable target state (e.g., Bell, GHZ, or arbitrary user-defined states), with support for both dense (per-step fidelity improvement) and sparse (terminal fidelity threshold) reward schemes.

Table 3: Built-in silicon spin qubit device profiles in SiliQun. All parameters are drawn from the experimental references cited in the text.

Parameter	Donor (P:Si)	SiMOS (QD)	GAA (Nanowire)
T_1 relaxation	30 s ^a	10 s ^b	5 s ^c
T_2^* dephasing	0.5 ms ^a	20 μ s ^d	10 μ s ^c
T_2 (Hahn echo)	1.2 ms ^a	100 μ s ^d	50 μ s ^c
1Q gate time	1 μ s	200 ns	50 ns
2Q gate time	100 ns	50 ns	30 ns
1Q gate fidelity	99.99% ^e	99.93% ^d	99.9% ^f
2Q gate fidelity	99.9% ^g	99.65% ^h	$\sim$ 99% ^f
Readout fidelity	99.4%	98.5%	97.5%
Charge noise	0.5 μ eV	2 μ eV ⁱ	3 μ eV ⁱ
Qubit spacing	15 nm	80 nm	60 nm

^aMuhonen et al. [2]; ^bLaucht et al. [9]; ^cProjected from Geyer et al. [10];

^dYoneda et al. [4]; ^eMuhonen et al. [3]; ^fGeyer et al. [10];

^gMadzik et al. [8]; ^hXue et al. [6]; ⁱConnors et al. [11].

2.2. Software functionalities

Beyond the core simulation loop, SiliQun provides several features designed to support practical DRL research workflows:

- **Configurable bond dimension:** Users can set $\chi_{\max}$ to control the accuracy–speed tradeoff. For DRL training where approximate dynamics suffice, small bond dimensions ($\chi \leq 16$) provide substantial speedups.
- **HPC runner:** The `hpc.runner` module generates PBS job scripts with configurable resource requests, automatically handles checkpointing of training state, and supports array jobs for parallel hyperparameter sweeps.
- **Extensible gate set:** New gates can be added by providing their unitary matrix; the tensor network layer automatically constructs the corresponding MPO.
- **Reproducibility:** All random number generators (for noise, initial states, and action sampling) are seeded through the Gymnasium API, ensuring fully reproducible training runs.

3. Illustrative examples

We present two illustrative examples that demonstrate SiliQun’s core functionality: a basic DRL training loop and a comprehensive performance characterization.

3.1. Example 1: DRL training loop

The following code snippet shows how to set up a SiliQun environment and connect it to a Stable-Baselines3 PPO agent for training a Bell state preparation policy on a noisy SiMOS device:

```
1 from siliqun.engine.gym_env import SiliQunEnv
2 from siliqun.engine.simulator import SimConfig
3 from stable_baselines3 import PPO
4
5 config = SimConfig(noise_enabled=True, max_bond_dim=16)
6 env = SiliQunEnv(
7     device="simos", n_qubits=2,
8     target_state="bell", config=config,
9     reward_type="dense"
10 )
11
12 model = PPO("MlpPolicy", env, verbose=1,
13             learning_rate=3e-4, n_steps=2048)
14 model.learn(total_timesteps=500_000)
15
16 # Evaluate learned policy
17 obs, _ = env.reset()
18 done = False
19 while not done:
20     action, _ = model.predict(obs, deterministic=True)
21     obs, reward, terminated, truncated, info = env.step(action)
22     done = terminated or truncated
23 print(f"Final fidelity: {info['fidelity']:.4f}")
```

This example requires only 15 lines of user code to go from environment configuration to a trained policy, illustrating the low barrier to entry that SiliQun provides.

3.2. Example 2: Performance benchmarks

We benchmarked SiliQun on a compute node of the Aziz Supercomputer at King Abdulaziz University, equipped with an Intel Xeon E5-2695 v2 processor (2.40 GHz, 24 cores) and 96 GB RAM, using the NumPy backend on a single core.

Throughput scaling. Figure 2 shows the simulation throughput as a function of qubit count. SiliQun sustains approximately 91 000 steps per second

across all tested system sizes from 2 to 16 qubits, demonstrating the flat scaling characteristic of tensor-network methods. This stands in sharp contrast to statevector simulators, whose throughput degrades exponentially with qubit count due to the $O(2^n)$ memory and computation requirements.

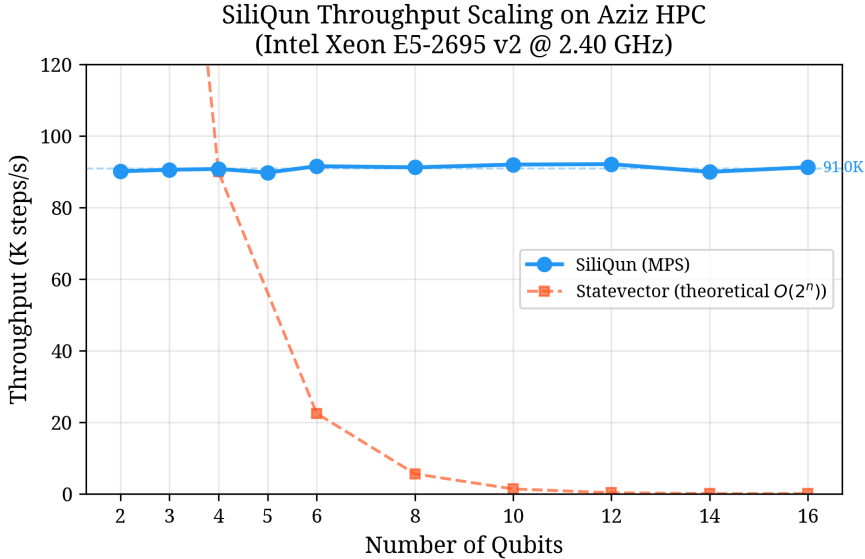


Figure 2: Simulation throughput (steps per second) as a function of the number of qubits on the Aziz HPC (Intel Xeon E5-2695 v2 @ 2.40 GHz). The MPS-based backend maintains nearly constant throughput (~ 91 K steps/s) from 2 to 16 qubits, while the theoretical statevector scaling degrades exponentially.

Comparison with existing tools. To contextualize SiliQun’s performance, we compared its gate-level throughput against Qiskit Aer v0.17.2 (statevector method) and QuTiP v5.2.3 (full state vector via Qobj multiplication). All three tools applied the same workload: Hadamard gates in round-robin across all qubits. Figure 3 presents the results. SiliQun achieves $5.5\times$ higher throughput than Qiskit Aer at 2 qubits, growing to $29\times$ at 14 qubits. Against QuTiP, the advantage reaches $41\times$ at 14 qubits. We emphasize that this comparison reflects the throughput advantage of MPS-based approximate simulation over exact statevector methods; for highly entangled states, the MPS representation would require larger bond dimensions and correspondingly lower throughput.

Noise overhead. Enabling the full noise model (amplitude damping, phase damping, and charge noise) introduces an overhead of approximately 20–39% depending on system size, as shown in Figure 4. Even with noise enabled, throughput remains above 69 000 steps per second for all tested system sizes, comfortably exceeding the throughput floor required for practical DRL train-

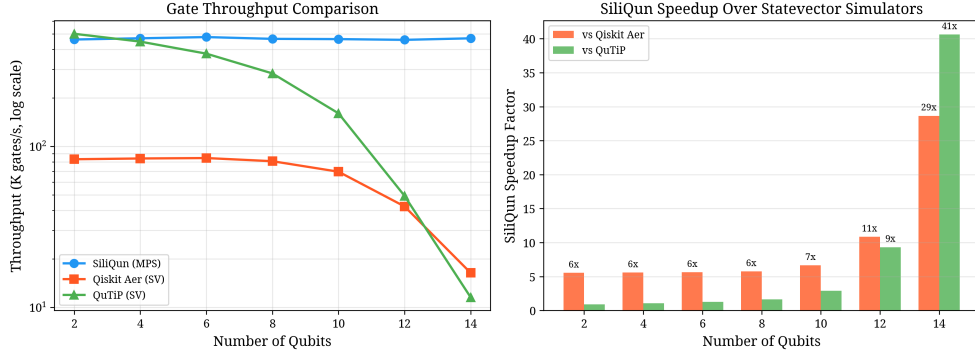


Figure 3: Left: Gate throughput comparison between SiliQun (MPS), Qiskit Aer (statevector), and QuTiP (statevector) on a log scale. Right: SiliQun’s speedup factor over both statevector simulators. The MPS advantage grows with system size as statevector methods encounter exponential scaling.

ing.

Bond dimension scaling. Figure 5 shows that throughput remains nearly constant across bond dimensions from $\chi = 2$ to $\chi = 64$ for an 8-qubit system. This indicates that the computational bottleneck lies in the gate application overhead rather than the SVD truncation, and that users can increase simulation accuracy by raising χ without significant performance penalty in this regime.

Gate fidelity. Gate fidelity was validated by comparing SiliQun’s output against analytical expectations for a suite of single-qubit rotations (R_x , R_y , R_z , Hadamard), two-qubit entangling gates (CNOT, CZ, $\sqrt{\text{SWAP}}$), and entanglement entropy calculations. All tests passed with errors at or below machine precision ($\leq 2.2 \times 10^{-16}$), confirming the correctness of the tensor-network implementation.

4. Impact

SiliQun addresses a practical bottleneck in the emerging field of DRL-based quantum control. Before SiliQun, a researcher wishing to train a DRL agent for silicon spin qubit control would need to either build a custom simulator from scratch—a task requiring expertise in both quantum physics and software engineering—or adapt a general-purpose quantum simulator that lacks silicon-specific physics and DRL compatibility. SiliQun eliminates this barrier by providing a ready-to-use, benchmarked, and extensible platform. The software has several concrete impacts on the research community. First, it enables **rapid prototyping** of DRL algorithms for quantum control: researchers can evaluate new policy architectures, reward shaping strategies,

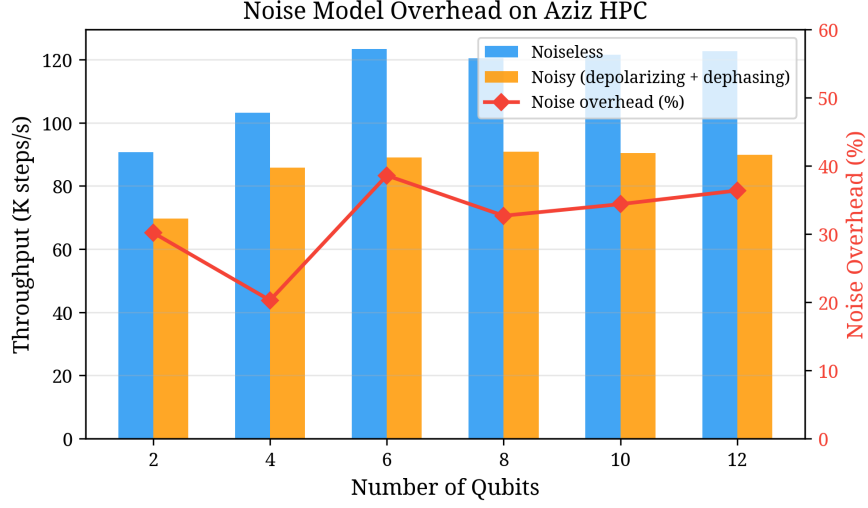


Figure 4: Comparison of simulation throughput with and without noise channels enabled on the Aziz HPC. The noise overhead (red diamonds, right axis) ranges from 20% to 39%, averaging approximately 32%.

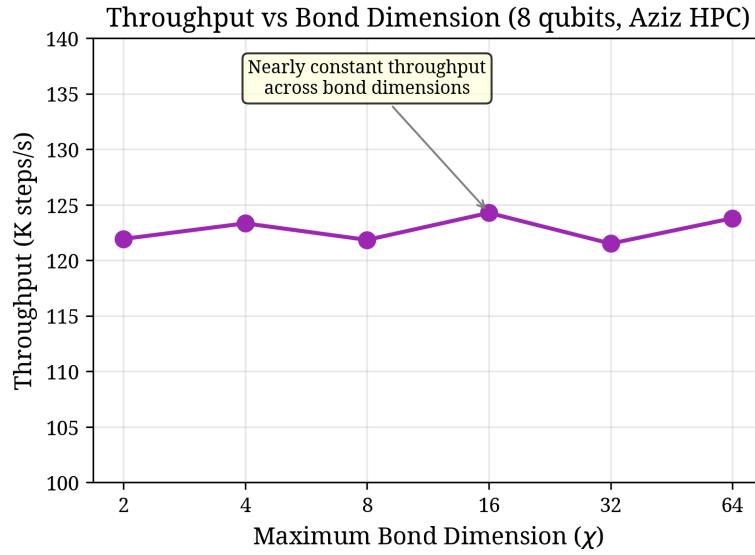


Figure 5: Throughput as a function of maximum bond dimension χ for an 8-qubit system on the Aziz HPC. Throughput remains approximately constant at ~ 122 K steps/s across all tested bond dimensions.

or curriculum learning approaches in hours rather than weeks. Second, the tensor-network backend enables **scalability studies** on systems of 10–20 qubits that would be intractable for full state-vector simulators, which is essential for understanding how DRL policies scale with system size. Third, the built-in device profiles facilitate **hardware co-design**: by training the same DRL algorithm on different device profiles, researchers can quantify how hardware characteristics (e.g., coherence times, gate speeds) affect the learnability of control policies, providing actionable feedback for device engineers.

SiliQun already serves as the foundational simulation layer for ongoing research into robust quantum control using the DYNAMO (DYNamic Adaptive Multi-objective Orchestration) framework, where it provides the training environment for DRL agents that learn to mitigate noise in silicon spin qubit arrays.

5. Conclusions

We have presented SiliQun, an open-source tensor-network simulator purpose-built for training deep reinforcement learning agents on silicon spin qubit control tasks. By combining the computational efficiency of Matrix Product States with a native Gymnasium interface and physically motivated device profiles calibrated to the experimental literature, SiliQun occupies a unique position in the quantum simulation landscape—bridging the gap between detailed physics simulators and the high-throughput requirements of modern DRL training.

Benchmarks on the Aziz Supercomputer demonstrate that SiliQun sustains approximately 91 000 simulation steps per second across 2–16 qubit systems, achieving 6–29 $\times$ higher throughput than Qiskit Aer’s statevector simulator. The noise model adds a modest 32% overhead on average, and gate fidelity is validated to machine precision.

While SiliQun currently targets the linear chain topologies characteristic of early-stage silicon spin qubit arrays, scaling to 100-qubit 2D architectures for fault-tolerant operations will require expanding beyond the 1D constraints of Matrix Product States. Future development will focus on three critical directions to overcome these scalability bottlenecks. First, to handle the massive entanglement of a 100-qubit 2D grid, the tensor-network core will be upgraded to support multi-dimensional geometries, such as Tree Tensor Networks (TTNs) [27], Tensor Rings (TR) [28], and Projected Entangled Pair States (PEPS) [29]. Integration with generalized tensor-network libraries like quimb [30], as recently demonstrated by the SpinPulse package [21], will facilitate this transition. Second, to manage the entanglement overhead

during DRL training, the framework will implement native Message Passing Interface (MPI) support via libraries such as NVIDIA’s `cuTensorNet` [31], enabling distributed tensor contractions across supercomputing clusters like Aziz. Finally, we will incorporate recent experimental breakthroughs in MPO tomography [32], which demonstrated that MPO can efficiently represent and mitigate errors in highly entangled 96-qubit mixed states. This will allow SiliQun to directly simulate and learn from realistic decoherence within the tensor network, rather than relying solely on post-hoc CPTP channels.

Acknowledgements

The author acknowledges the use of the Aziz Supercomputer at King Abdulaziz University for computational benchmarking.

References

- [1] Zwanenburg, F. A., Dzurak, A. S., Morello, A., Simmons, M. Y., Hollenberg, L. C., Klimeck, G., Rogge, S., Coppersmith, S. N., & Eriksson, M. A. (2013). Silicon quantum electronics. *Reviews of Modern Physics*, 85(3), 961–1019.
- [2] Muhonen, J. T., Dehollain, J. P., Laucht, A., Hudson, F. E., Kalra, R., Sekiguchi, T., Itoh, K. M., Jamieson, D. N., McCallum, J. C., Dzurak, A. S., & Morello, A. (2014). Storing quantum information for 30 seconds in a nanoelectronic device. *Nature Nanotechnology*, 9(12), 986–991.
- [3] Muhonen, J. T., Laucht, A., Simmons, S., Dehollain, J. P., Kalra, R., Hudson, F. E., Freer, S., Itoh, K. M., Jamieson, D. N., McCallum, J. C., Dzurak, A. S., & Morello, A. (2015). Quantifying the quantum gate fidelity of single-atom spin qubits in silicon by randomized benchmarking. *Journal of Physics: Condensed Matter*, 27(15), 154205.
- [4] Yoneda, J., Takeda, K., Otsuka, T., Nakajima, T., Delbecq, M. R., Allison, G., Honda, T., Kadera, T., Oda, S., Hoshi, Y., Usami, N., Itoh, K. M., & Tarucha, S. (2018). A quantum-dot spin qubit with coherence limited by charge noise and fidelity higher than 99.9%. *Nature Nanotechnology*, 13(2), 102–106.
- [5] Noiri, A., Takeda, K., Nakajima, T., Kobayashi, T., Sammak, A., Scappucci, G., & Tarucha, S. (2022). Fast universal quantum gate above the fault-tolerance threshold in silicon. *Nature*, 601(7893), 338–342.

- [6] Xue, X., Russ, M., Samkharadze, N., Undseth, B., Sammak, A., Scappucci, G., & Vandersypen, L. M. K. (2022). Quantum logic with spin qubits crossing the surface code threshold. *Nature*, 601(7893), 343–347.
- [7] Mills, A. R., Guinn, C. R., Gullans, M. J., Sigillito, A. J., Feldman, M. M., Nielsen, E., & Petta, J. R. (2022). Two-qubit silicon quantum processor with operation fidelity exceeding 99%. *Science Advances*, 8(14), eabn5130.
- [8] Madzik, M. T., Asaad, S., Youssry, A., Joecker, B., Rudinger, K. M., Nielsen, E., Young, K. C., Proctor, T. J., Baczewski, A. D., Laucht, A., Schmitt, V., Hudson, F. E., Itoh, K. M., Jakob, A. M., Johnson, B. C., Jamieson, D. N., Dzurak, A. S., Ferrie, C., Blume-Kohout, R., & Morello, A. (2022). Precision tomography of a three-qubit donor quantum processor in silicon. *Nature*, 601(7893), 348–353.
- [9] Laucht, A., Muhonen, J. T., Mohiyaddin, F. A., Kalra, R., Dehollain, J. P., Freer, S., Hudson, F. E., Veldhorst, M., Rahman, R., Klimeck, G., Itoh, K. M., Jamieson, D. N., McCallum, J. C., Dzurak, A. S., & Morello, A. (2015). Electrically controlling single-spin qubits in a continuous microwave field. *Science Advances*, 1(3), e1500022.
- [10] Geyer, S., Hetényi, B., Bosco, S., Camenzind, L. C., Eggli, R. S., Fuhrer, A., Loss, D., Warburton, R. J., Zumbühl, D. M., & Kuhlmann, A. V. (2022). Two-qubit logic with anisotropic exchange in a fin field-effect transistor. *arXiv preprint arXiv:2212.02308*.
- [11] Connors, E. J., Nelson, J., Edge, L. F., & Nichol, J. M. (2022). Charge-noise spectroscopy of Si/SiGe quantum dots via dynamically-decoupled exchange oscillations. *Nature Communications*, 13(1), 940.
- [12] Struck, T., Hollmann, A., Schauer, F., Fedorets, O., Schmidbauer, A., Sawano, K., Riemann, H., Abrosimov, N. V., Cywiński, Ł., Bougeard, D., & Schreiber, L. R. (2020). Low-frequency spin qubit energy splitting noise in highly purified $^{28}\text{Si}/\text{SiGe}$. *npj Quantum Information*, 6(1), 40.
- [13] Khaneja, N., Reiss, T., Schulte-Herbrüggen, T., & Glaser, S. J. (2005). Optimal control of coupled spin dynamics: design of NMR pulse sequences by gradient ascent algorithms. *Journal of Magnetic Resonance*, 172(2), 296–305.
- [14] Niu, M. Y., Boixo, S., Smelyanskiy, V. N., & Neven, H. (2019). Universal quantum control through deep reinforcement learning. *npj Quantum Information*, 5(1), 33.

- [15] Fösel, T., Tighineanu, P., Weiss, T., & Marquardt, F. (2018). Reinforcement learning with neural networks for quantum feedback. *Physical Review X*, 8(3), 031084.
- [16] Porotti, R., Tamascelli, D., Restelli, M., & Prati, E. (2019). Coherent transport of quantum states by deep reinforcement learning. *Communications Physics*, 2(1), 61.
- [17] An, Z., & Zhou, D. L. (2019). Deep reinforcement learning for quantum gate control. *EPL (Europhysics Letters)*, 126(6), 60002.
- [18] Nanoacademic Technologies Inc. (2023). QTCAD: Quantum Technology Computer-Aided Design. <https://www.nanoacademic.com/qtcad>
- [19] Qiskit contributors. (2024). Qiskit: An open-source framework for quantum computing. <https://github.com/Qiskit/qiskit>
- [20] Cirq Developers. (2018). Cirq: A Python framework for creating, editing, and invoking noisy intermediate scale quantum circuits. <https://github.com/quantumlib/Cirq>
- [21] Vermersch, B., et al. (2026). SpinPulse: A pulse-level simulator for spin qubit architectures. *arXiv preprint arXiv:2601.10435*.
- [22] Bergholm, V., Izaac, J., Schuld, M., et al. (2018). PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*.
- [23] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1–8.
- [24] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J., Jordan, M., & Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In *International Conference on Machine Learning* (pp. 3053–3062).
- [25] Stano, P. & Loss, D. (2024). Review of performance metrics of spin qubits in gated semiconducting nanostructures. *arXiv preprint arXiv:2107.06485v7*.
- [26] Stemp, H. G., Asaad, S., Madzik, M. T., et al. (2025). An 11-qubit atom processor in silicon. *Nature*, published online December 17, 2025.

- [27] Shi, Y. Y., Duan, L. M., & Vidal, G. (2006). Classical simulation of quantum many-body systems with a tree tensor network. *Physical Review A*, 74(2), 022320.
- [28] Zhao, Q., Zhou, G., Xie, S., Zhang, L., & Cichocki, A. (2016). Tensor ring decomposition. *arXiv preprint arXiv:1606.05535*.
- [29] Verstraete, F., & Cirac, J. I. (2004). Renormalization algorithms for quantum-many body systems in two and higher dimensions. *arXiv preprint cond-mat/0407066*.
- [30] Gray, J. (2018). quimb: A python library for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29), 819.
- [31] NVIDIA Corporation. (2024). cuQuantum: cuTensorNet. <https://developer.nvidia.com/cuquantum-sdk>
- [32] Votto, M., Ljubotina, M., Lancien, C., Cirac, J. I., Zoller, P., Serbyn, M., Piroli, L., & Vermersch, B. (2026). Learning mixed quantum states in large-scale experiments. *Physical Review Letters*, 136(9), 090801.